LAPORAN PROYEK SISTEM TERDISTRIBUSI
NUSANTARA CINEMA

"Sistem Reservasi Tiket Bioskop Terdistribusi dengan Replikasi Master-Slave dan Two-Phase Commit"



Disusun oleh:

Muhammad Rafly Aprial Ridho	(105841120324)
Emil Rahmat	(105841119124)
Tiara Nabila Rajab	(105841119224)
Ahmad Ali Dwil Zaydi	(105841120224)

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH MAKASSAR

2026

DAFTAR ISI

DAFTAR ISI ii
BAB I 1
PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 2
1.3 Tujuan & Manfaat 2
1.4 Ruang Lingkup Sistem 2
BAB II 3
LANDASAN TEORI 3
2.1 Konsep Sistem Terdistribusi 3
2.2 Replikasi Data (Master-Slave) & Two-Phase Commit / Mutual Exclusion 3
2.3 Teknologi yang Digunakan 3
BAB III 5
ANALISIS DAN PERANCANGAN SISTEM 5
3.1 Deskripsi & Arsitektur Sistem 5
3.2 Analisis Kebutuhan Fungsional & Non-Fungsional 7
3.3 Konsep Sistem Terdistribusi yang Diterapkan 7
3.4 Struktur Folder Proyek & Peran Setiap File 8
BAB IV 9
IMPLEMENTASI DAN PENGUJIAN SISTEM 9
4.1 Implementasi Kode Program 9
4.2 Antarmuka Web Dashboard 16
4.3 Pengujian Sistem 17
4.4 Hasil Pengujian 18
BAB V 19
PENUTUP 19
5.1 Kesimpulan 19

5.2 Saran Pengembangan 20

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi dewasa ini mendorong penyelenggaraan berbagai layanan secara tersebar di berbagai lokasi geografis. Fenomena ini sangat nampak pada industri hiburan, khususnya bisnis bioskop yang umumnya mengoperasikan puluhan hingga ratusan cabang di berbagai kota. Setiap cabang menawarkan jadwal tayang, harga tiket, dan ketersediaan kursi yang dinamis, sehingga diperlukan sebuah sistem yang mampu mengelola data secara terpadu namun tetap tanggap terhadap kebutuhan lokal tiap cabang. Kebutuhan akan sistem reservasi tiket yang handal, cepat, dan konsisten antar cabang menjadi tantangan tersendiri bagi para pengelola jaringan bioskop modern.

Apabila seluruh cabang bergantung sepenuhnya pada satu server pusat tanpa adanya mekanisme terdistribusi, maka kegagalan (failure) pada server tunggal tersebut akan melumpuhkan seluruh layanan di semua cabang sekaligus. Selain itu, lonjakan permintaan (request) secara bersamaan pada saat film sedang tren dapat membuat server pusat kewalahan dan mengakibatkan penundaan bahkan kehilangan transaksi. Oleh karena itu, diperlukan sebuah arsitektur sistem terdistribusi yang mampu menjaga konsistensi data harga tiket antar cabang, membagi beban kerja, serta menjamin keutuhan (atomicity) setiap transaksi pemesanan meskipun terjadi akses secara konkuren.

Proyek Nusantara Cinema merupakan simulasi sistem reservasi tiket bioskop terdistribusi yang dirancang untuk mendemonstrasikan konsep-konsep dasar tersebut secara konkret. Sistem ini terdiri dari tiga server cabang, yaitu Jakarta sebagai server pusat (Master) serta Makassar dan Surabaya sebagai server cabang (Slave). Untuk menjaga keseragaman harga, sistem mengimplementasikan replikasi data Master-Slave dengan dua mode penyebaran, yakni serentak (sync) dan bertahap (async). Sementara itu, untuk menjamin bahwa sebuah pemesanan

tiket benar-benar terselesaikan secara utuh, sistem menggunakan protokol transaksi Two-Phase Commit (2PC) yang melibatkan tiga layanan: pembayaran, pengelolaan kursi, dan penerbitan tiket. Guna mencegah kondisi balapan (race condition) saat dua pemesanan datang secara bersamaan, diterapkan pula mekanisme mutual exclusion (lock). Melalui simulasi ini, diharapkan pembaca dapat memahami bagaimana konsep sistem terdistribusi diaplikasikan dalam kasus nyata pemesanan tiket bioskop.

1.2 Rumusan Masalah

- Bagaimana menjaga konsistensi data harga tiket antar cabang yang tersebar?
- Bagaimana menjamin keutuhan (atomicity) transaksi pemesanan yang melibatkan beberapa layanan?
- Bagaimana mencegah kondisi balapan (race condition) pada pemesanan yang datang bersamaan?

1.3 Tujuan & Manfaat

- Membangun simulasi jaringan bioskop terdistribusi dengan tiga node server.
- Mengimplementasikan replikasi data Master-Slave dengan mode sinkron dan asinkron.
- Mengimplementasikan protokol Two-Phase Commit untuk transaksi pemesanan tiket.
- Menjamin konsistensi data melalui mekanisme mutual exclusion (lock).
- Memberikan pemahaman praktis mengenai konsep sistem terdistribusi bagi pembaca.

1.4 Ruang Lingkup Sistem

Sistem dibangun sebagai simulasi menggunakan bahasa Python dengan kerangka kerja Flask. Latensi jaringan pada replikasi asinkron disimulasikan menggunakan penundaan waktu (threading), bukan keterlambatan jaringan fisik yang sebenarnya. Seluruh node berjalan dalam satu proses aplikasi untuk tujuan demonstrasi. Fitur yang dicakup meliputi multi-cabang, denah kursi interaktif, pemesanan tiket via 2PC, panel admin, serta monitor real-time.

BAB II
LANDASAN TEORI

2.1 Konsep Sistem Terdistribusi

Sistem terdistribusi adalah kumpulan komputer independen yang bagi pengguna tampak sebagai satu sistem koheren dan tunggal. Ciri utamanya meliputi berbagi sumber daya, keterbukaan, konkurensi, skalabilitas, dan toleransi terhadap kegagalan. Tantangan utamanya adalah menjaga konsistensi data dan menangani kegagalan sebagian (partial failure).

2.2 Replikasi Data (Master-Slave) & Two-Phase Commit / Mutual Exclusion

Replikasi adalah penyalinan data dari satu node (Master) ke satu atau lebih node lain (Slave) agar data tersedia di beberapa lokasi. Pada model Master-Slave, seluruh pembaruan data dilakukan pada server pusat, kemudian disebarkan ke server cabang. Terdapat dua pendekatan penyebaran:

- Replikasi Sinkron (Sync): server pusat menunggu konfirmasi dari seluruh cabang sebelum transaksi selesai, sehingga data langsung seragam namun lebih lambat.
- Replikasi Asinkron (Async): server pusat menyelesaikan pembaruan lebih dulu, sedangkan cabang memperbarui diri secara bertahap sesuai latensi masing-masing, sehingga lebih cepat namun berpotensi terjadi ketidakseragaman sementara (eventual consistency).

Two-Phase Commit (2PC) adalah protokol untuk mencapai kesepakatan (agreement) dalam transaksi terdistribusi yang melibatkan beberapa partisipan, dipimpin oleh sebuah koordinator. Mutual Exclusion (lock) menjamin hanya satu proses yang dapat mengakses sumber daya bersama pada satu waktu, sehingga mencegah race condition.

2.3 Teknologi yang Digunakan

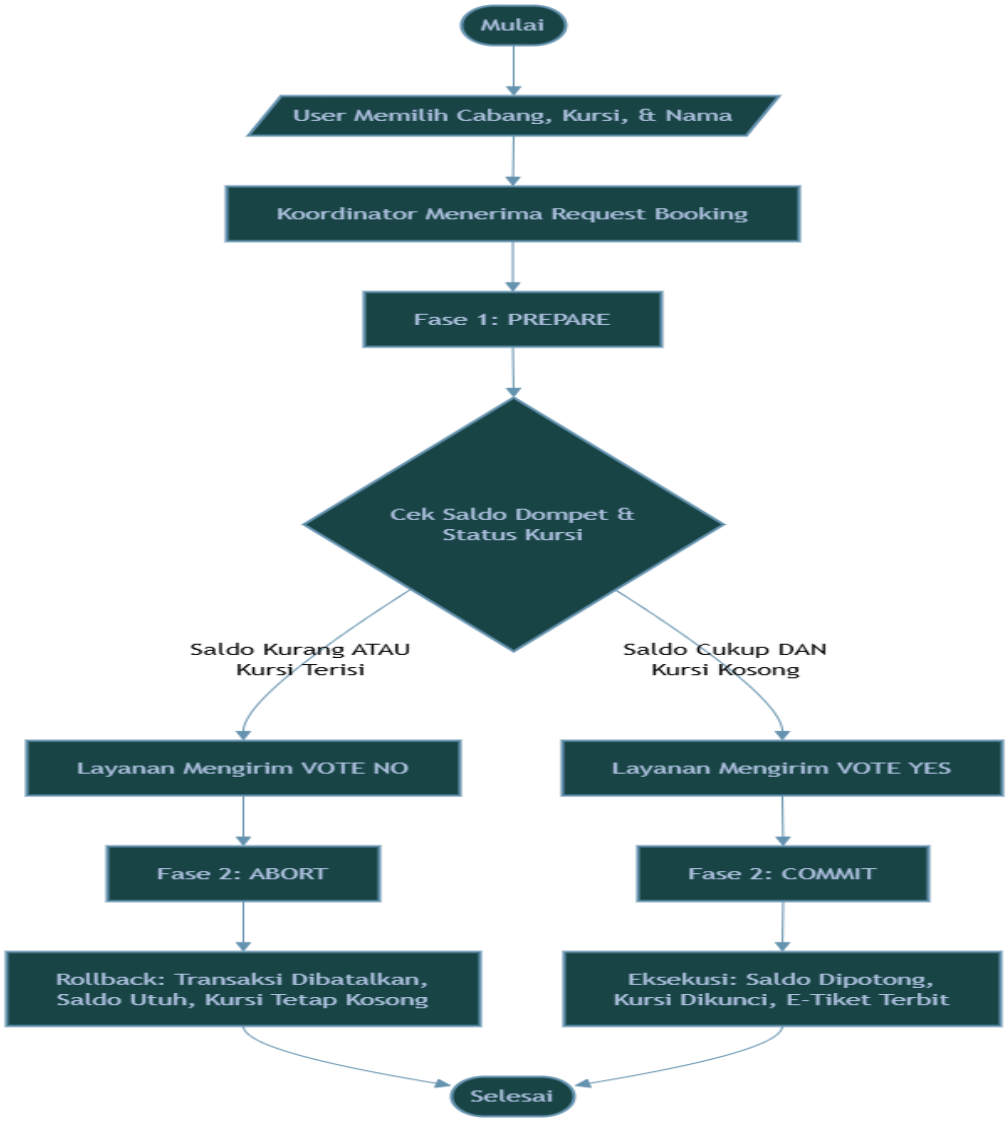
Teknologi	Peran dalam Sistem
Python 3.x	Bahasa pemrograman utama untuk logika sistem

Flask	Web framework untuk routing halaman & REST API
HTML / CSS / JavaScript	Antarmuka pengguna (front-end)
Threading (Python)	Simulasi latensi replikasi & lock transaksi
REST API (JSON)	Komunikasi data antara front-end dan back-end

BAB III
ANALISIS DAN PERANCANGAN SISTEM

3.1 Deskripsi & Arsitektur Sistem

Untuk memberikan gambaran yang lebih jelas mengenai bagaimana sistem menangani transaksi pemesanan agar tidak terjadi kesalahan data (*double booking* atau saldo terpotong tanpa tiket), sistem ini menerapkan algoritma *Two-Phase Commit* (2PC). Alur kerja dari algoritma pemesanan tiket tersebut dapat dilihat pada *flowchart* berikut:



Berdasarkan *flowchart* di atas, proses transaksi terdistribusi berjalan melalui tahapan-tahapan yang ketat untuk menjamin keutuhan data (*atomicity*):

1. **Tahap Inisiasi:** Proses dimulai ketika pengguna (*user*) mengirimkan permintaan pemesanan yang berisi nama, pilihan cabang, dan daftar kursi. Permintaan ini diterima oleh Koordinator Sistem (Server Utama).
2. **Fase 1 (PREPARE / Voting):** Koordinator tidak langsung memproses pesanan, melainkan masuk ke Fase 1. Koordinator akan menanyai dua layanan partisipan:
   - **Layanan Pembayaran** mengecek apakah saldo dompet pengguna mencukupi.
   - **Layanan Kursi** mengecek apakah kursi yang dipilih masih kosong dan belum diambil orang lain.
3. **Pemberian Suara (Voting):**
   - Jika saldo tidak cukup **ATAU** kursi sudah terisi, layanan akan mengirimkan status **VOTE NO** (Menolak).
   - Jika saldo cukup **DAN** kursi masih kosong, layanan akan mengirimkan status **VOTE YES** (Menyetujui).
4. **Fase 2 (Keputusan Akhir):** Berdasarkan hasil *voting* pada Fase 1, Koordinator mengambil keputusan mutlak:
   - **ABORT (Batal):** Jika ada minimal satu layanan yang membalas *VOTE NO*, sistem langsung membatalkan seluruh transaksi (*Rollback*). Saldo pengguna dijamin utuh dan status kursi tetap dikembalikan menjadi kosong.
   - **COMMIT (Sukses):** Jika semua layanan membalas *VOTE YES*, barulah sistem mengeksekusi perubahan secara permanen. Saldo dipotong, kursi resmi dikunci atas nama pemesan, dan e-tiket diterbitkan.

Dengan alur ini, sistem dipastikan bebas dari kecacatan data apabila terjadi masalah di tengah proses pemesanan.

3.2 Analisis Kebutuhan Fungsional & Non-Fungsional

Tabel berikut merangkum kebutuhan fungsional sistem:

No	Kebutuhan Fungsional	Keterangan
1	Menampilkan daftar cabang	Halaman utama menampilkan status 3 server
2	Memesan tiket	Pilih kursi (maks 6) & bayar via 2PC
3	Ubah harga tiket	Admin mengubah harga di pusat (sync/async)
4	Atur saldo dompet	Admin mengatur saldo pelanggan
5	Monitor sistem	Pantau log node, replikasi, & riwayat
6	Reset data	Mengembalikan seluruh data ke kondisi awal

Tabel berikut merangkum kebutuhan non-fungsional sistem:

No	Kebutuhan Non-Fungsional	Keterangan
1	Konsistensi	Harga seragam (sync) / eventual (async) antar cabang
2	Atomicity	Transaksi 2PC: sukses seluruhnya atau gagal seluruhnya
3	Concurrency	Lock mencegah dua pemesanan ambil kursi sama
4	Usability	Antarmuka web sederhana & responsif
5	Maintainability	Kode terbagi per modul (bioskop, replikasi, 2PC)

3.3 Konsep Sistem Terdistribusi yang Diterapkan

Node saling berkomunikasi melalui koordinator tunggal (app.py) yang menjadi titik temu seluruh operasi. Pada replikasi, server pusat Jakarta menulis perubahan ke replication log, lalu menyebarkannya ke cabang. Pada mode sync, pusat menunggu seluruh cabang selesai; pada mode async, penyebaran dilakukan melalui thread terpisah dengan latensi Makassar ±4 detik dan Surabaya ±8 detik. Pada transaksi pemesanan, koordinator meminta vote dari tiga layanan (Pembayaran, Kursi, Tiket) lalu memutuskan COMMIT atau ABORT berdasarkan hasil voting.

3.4 Struktur Folder Proyek & Peran Setiap File

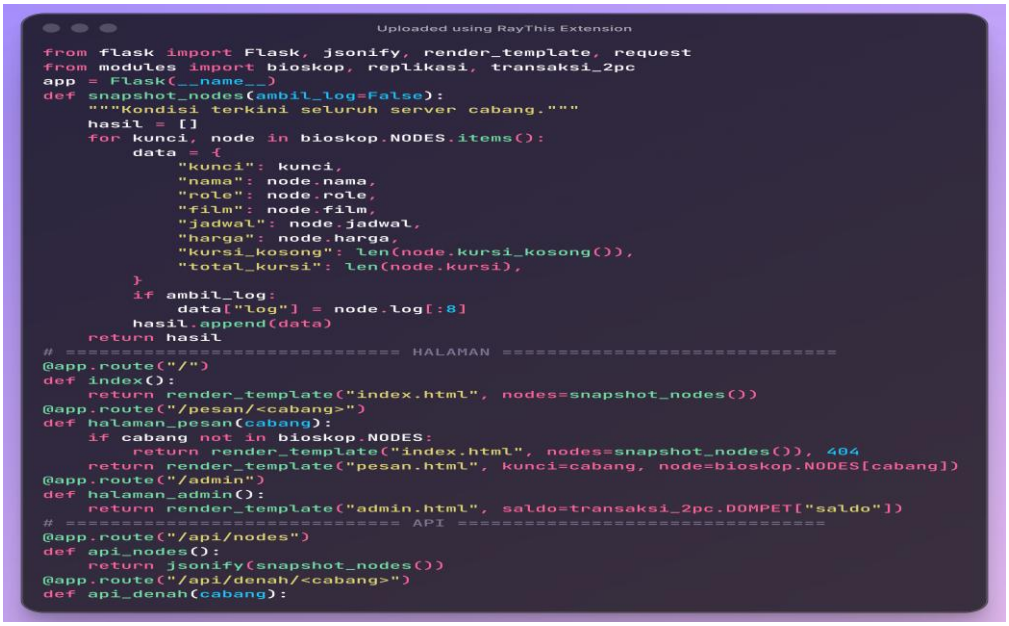
Berkas / Folder	Peran
app.py	Entry point Flask: routing halaman & endpoint API
modules/__init__.py	Penanda paket Python modules
modules/bioskop.py	Data node server & denah kursi (NodeServer)
modules/replikasi.py	Replikasi harga Master-Slave (sync/async)
modules/transaksi_2pc.py	Transaksi pemesanan Two-Phase Commit
templates/	Antarmuka pengguna (HTML Jinja2)
static/style.css	Berkas gaya (CSS) antarmuka
requirements.txt	Daftar dependensi (flask)

BAB IV
IMPLEMENTASI DAN PENGUJIAN SISTEM

4.1 Implementasi Kode Program

1. app.py Entry Point & Routing Flask

File app.py merupakan titik masuk (entry point) aplikasi dan berjalan di atas kerangka Flask. Pada bagian atas, dilakukan impor modul serta pembuatan objek app = Flask(__name__) yang menjadi inti aplikasi web. Fungsi bantuan snapshot_nodes() berperan penting: ia melakukan iterasi terhadap dictionary NODES yang berisi ketiga server (Jakarta, Makassar, Surabaya), lalu mengambil data terkini tiap node seperti nama, peran (Master/Slave), film, jadwal, harga, jumlah kursi kosong, dan total kursi. Data ini dikembalikan dalam bentuk list of dictionary agar mudah dikirim ke template HTML maupun ke response JSON pada API. Selanjutnya, bagian routing halaman (HALAMAN) mendefinisikan tiga rute utama: '/' untuk beranda, '/pesan/<cabang>' untuk halaman pemesanan per cabang, dan '/admin' untuk panel administrasi. Setiap rute memanggil fungsi yang mengembalikan render_template() dengan data yang dibutuhkan.




```python
from flask import Flask, jsonify, render_template, request
from modules import bioskop, replikasi, transaksi_2pc
app = Flask(__name__)
def snapshot_nodes(ambil_log=False):
    """Kondisi terkini seluruh server cabang."""
    hasil = []
    for kunci, node in bioskop.NODES.items():
        data = {
            "kunci": kunci,
            "nama": node.nama,
            "role": node.role,
            "film": node.film,
            "jadwal": node.jadwal,
            "harga": node.harga,
            "kursi_kosong": len(node.kursi_kosong()),
            "total_kursi": len(node.kursi),
        }
        if ambil_log:
            data["log"] = node.log[:8]
        hasil.append(data)
    return hasil
# ============================== HALAMAN ==============================
@app.route("/")
def index():
    return render_template("index.html", nodes=snapshot_nodes())
@app.route("/pesan/<cabang>")
def halaman_pesan(cabang):
    if cabang not in bioskop.NODES:
        return render_template("index.html", nodes=snapshot_nodes()), 404
    return render_template("pesan.html", kunci=cabang, node=bioskop.NODES[cabang])
@app.route("/admin")
def halaman_admin():
    return render_template("admin.html", saldo=transaksi_2pc.DOMPET["saldo"])
# ============================== API ==============================
@app.route("/api/nodes")
def api_nodes():
    return jsonify(snapshot_nodes())
@app.route("/api/denah/<cabang>")
def api_denah(cabang):
```


Uploaded using RayThis Extension
def api_denah(cabang):
if cabang not in bioskop.NODES:
return jsonify({"error": "Cabang tidak dikenal"}), 404
node = bioskop.NODES[cabang]
return jsonify({"kursi": node.kursi, "harga": node.harga,
"saldo": transaksi_2pc.DOMPET["saldo"]})
@app.route("/api/booking", methods=["POST"])
def api_booking():
data = request.json
cabang = data.get("cabang")
daftar_kursi = data.get("kursi", [])
nama = (data.get("nama") or "").strip()[:30]
if cabang not in bioskop.NODES:
return jsonify({"error": "Cabang tidak dikenal"}), 404
if not nama:
return jsonify({"error": "Nama pemesan wajib diisi"}), 400
if not daftar_kursi:
return jsonify({"error": "Pilih minimal satu kursi terlebih dahulu"}), 400
if len(daftar_kursi) > 6:
return jsonify({"error": "Maksimal 6 kursi per transaksi"}), 400
hasil = transaksi_2pc.jalankan_2pc(cabang, daftar_kursi, nama)
hasil["denah"] = bioskop.NODES[cabang].kursi
return jsonify(hasil)
---- Panel admin ----
@app.route("/api/admin/harga", methods=["POST"])
def api_admin_harga():
data = request.json
try:
harga = int(data.get("harga", 0))
except (TypeError, ValueError):
return jsonify({"error": "Harga harus berupa angka"}), 400
if not 1000 <= harga <= 1000000:
return jsonify({"error": "Harga harus antara Rp1.000 - Rp1.000.000"}), 400
if data.get("mode") == "sync":
langkah = replikasi.update_harga_sync(harga)
else:
langkah = replikasi.update_harga_async(harga)
return jsonify({"langkah": langkah})
@app.route("/api/admin/saldo", methods=["POST"])
def api_admin_saldo():
transaksi_2pc.atur_saldo(request.json.get("saldo"))
return jsonify({"saldo": transaksi_2pc.DOMPET["saldo"]})
@app.route("/api/admin/reset", methods=["POST"])
def api_admin_reset():
bioskop.reset_semua()
replikasi.REPLICATION_LOG.clear()
transaksi_2pc.DOMPET["saldo"] = 200000
transaksi_2pc.RIWAYAT.clear()
return jsonify({"ok": True})
@app.route("/api/admin/monitor")
def api_admin_monitor():
return jsonify({
"nodes": snapshot_nodes(ambil_log=True),
"replication_log": replikasi.REPLICATION_LOG,
"riwayat": transaksi_2pc.RIWAYAT,
"saldo": transaksi_2pc.DOMPET["saldo"],
})
if __name__ == "__main__":
print("=" * 60)
print(" NUSANTARA CINEMA - Reservasi Tiket Bioskop")
print(" Buka browser: http://127.0.0.1:5001")
print("=" * 60)
app.run(debug=True, port=5001)

Pada bagian API, app.py menyediakan sekumpulan endpoint REST yang menjadi jangkar komunikasi antara antarmuka (front-end) dan logika sistem (back-end). Endpoint GET '/api/nodes' dan '/api/denah/<cabang>' berfungsi mengambil status serta denah kursi tiap cabang dalam format JSON. Endpoint POST '/api/booking' adalah pintu masuk pemesanan: ia membaca data JSON berupa cabang, daftar kursi, dan nama pemesan, melakukan validasi (cabang dikenal, nama tidak kosong, minimal 1 dan maksimal 6 kursi), lalu memanggil transaksi_2pc.jalankan_2pc() yang menjalankan protokol Two-Phase Commit. Endpoint '/api/admin/harga' memanggil replikasi (sync/async) untuk mengubah harga, '/api/admin/saldo' mengatur saldo dompet, '/api/admin/reset' mengembalikan seluruh data, dan '/api/admin/monitor' menyajikan data pemantauan (log node, replication log, riwayat transaksi, saldo). Semua respons dikembalikan dengan jsonify() sehingga dapat dikonsumsi oleh JavaScript di sisi klien secara asinkron.

2. **modules/bioskop.py Data Node & Denah Kursi**

   Modul bioskop.py menjadi fondasi data sistem terdistribusi ini. Konstanta BARIS = ['A','B','C','D'] dan KOLOM = 6 menghasilkan 24 kursi per studio melalui fungsi buat_denah() yang mengembalikan dictionary dengan kunci 'A1'..'D6' bernilai None (kosong) atau string nama pemesan (terisi). Kelas NodeServer merepresentasikan satu server cabang: atributnya mencakup nama, role (Master/Slave), film, jadwal, harga, timestamp update_terakhir, denah kursi, dan list log. Method reset() mengembalikan node ke kondisi awal (harga Rp45.000, film default, denah kosong), method catat() menyimpan log berformat waktu+pesan (maksimal 20 entri terakhir), dan method kursi_kosong() mengembalikan daftar kursi yang masih None. Di akhir, dictionary global NODES diisi dengan tiga objek: Jakarta sebagai Master (pusat) serta Makassar dan Surabaya sebagai Slave. Fungsi reset_semua() memanggil reset() pada seluruh node untuk keperluan reset sistem.



3. modules/replikasi.py Replikasi Master-Slave

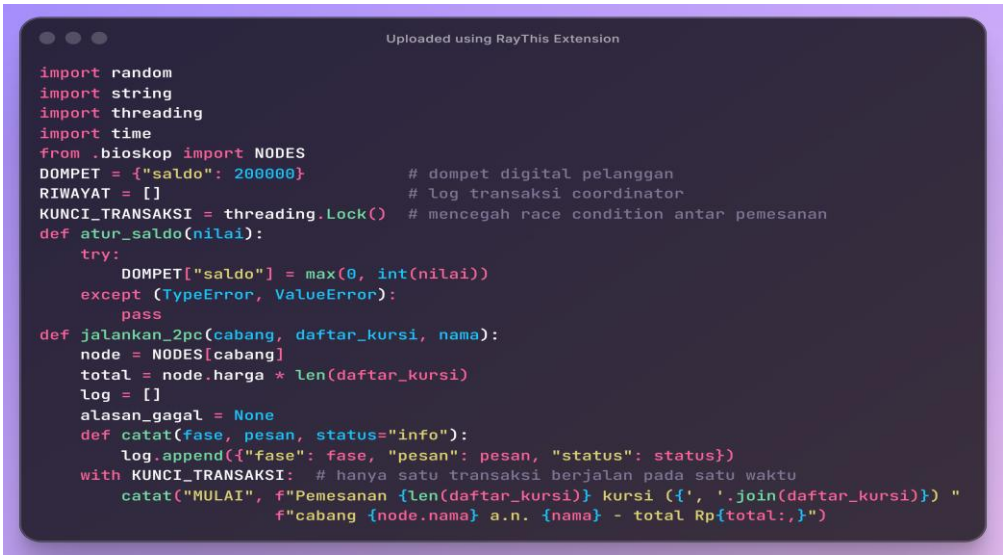
Modul replikasi.py mengimplementasikan mekanisme replikasi data Master-Slave untuk menyebarkan perubahan harga dari server pusat ke cabang. Variabel REPLICATION_LOG mencatat seluruh operasi perubahan, sedangkan LAG_MAKASSAR = 4 dan LAG_SURABAYA = 8 menyimpan simulasi latensi jaringan tiap cabang dalam detik. Fungsi update_harga_sync() (mode serentak) mengubah harga pada Master Jakarta, mencatat log, lalu dalam satu loop langsung menerapkan harga yang sama ke Makassar dan Surabaya serta mencatat konfirmasi tiap cabang; sehingga saat fungsi kembali, seluruh

node sudah seragam (strong consistency). Sebaliknya, fungsi update_harga_async() (mode bertahap) hanya memperbarui Master terlebih dulu, kemudian menjalankan dua threading.Thread terpisah (daemon) yang menunggu masing-masing 4 dan 8 detik sebelum menerapkan harga ke cabang. Pendekatan ini meniru eventual consistency di dunia nyata, di mana cabang sempat berbeda harga sebelum akhirnya menyamai pusat.



4. **modules/transaksi_2pc.py Two-Phase Commit**

Modul transaksi_2pc.py adalah inti dari jaminan atomicity pemesanan. Variabel global DOMPET menyimpan saldo dompet digital pelanggan (awal Rp200.000), RIWAYAT mencatat hasil tiap transaksi (maksimal 10), dan KUNCI_TRANSAKSI adalah threading.Lock yang melindungi seluruh proses agar hanya satu transaksi yang berjalan pada satu waktu (mutual exclusion, mencegah race condition). Fungsi jalankan_2pc(cabang, daftar_kursi, nama) membungkus logikanya dalam 'with KUNCI_TRANSAKSI'. Pada Fase 1 (PREPARE/voting), koordinator meminta vote dari tiga layanan: Layanan Pembayaran memeriksa kecukupan saldo, Layanan Kursi memvalidasi kursi tidak dikenal/terisi, dan Layanan Tiket selalu menjawab YES. Pada Fase 2 (COMMIT/ABORT), jika seluruh vote bernilai YES maka dikirim COMMIT: saldo dipotong, kursi dikunci atas nama pemesan, e-tiket berkode acak diterbitkan, dan node mencatat log sukses. Jika ada satu saja vote NO, dikirim ABORT dan seluruh perubahan dibatalkan (rollback) — saldo tetap utuh, kursi tetap kosong, tiket tidak diterbitkan. Hasil dikembalikan berupa dictionary lengkap (sukses, keputusan, votes, log, tiket, total, dll).

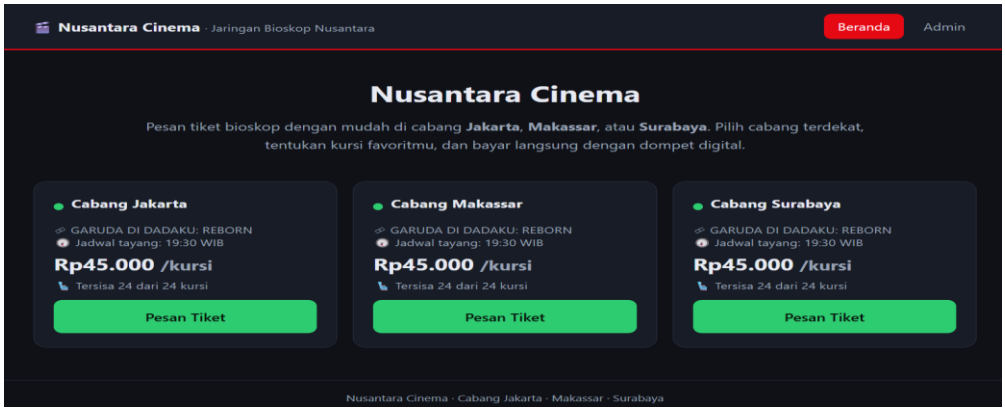



Uploaded using RayThis Extension
---------- FASE 1: PREPARE (VOTING) ----------
catat("PREPARE", "Coordinator meminta kesiapan seluruh layanan")
votes = {}
Layanan Pembayaran
if DOMPET["saldo"] >= total:
votes["Layanan Pembayaran"] = "YES"
catat("PREPARE", f"Layanan Pembayaran: saldo Rp{DOMPET['saldo']:,} mencukupi -> SIAP", "yes")
else:
votes["Layanan Pembayaran"] = "NO"
alasan_gagal = (f"Saldo dompet tidak mencukupi "
f"(butuh Rp{total:,}, saldo Rp{DOMPET['saldo']:,})")
catat("PREPARE", f"Layanan Pembayaran: {alasan_gagal} -> TOLAK", "no")
Layanan Kursi
tidak_dikenal = [k for k in daftar_kursi if k not in node.kursi]
terisi = [k for k in daftar_kursi if node.kursi.get(k) is not None]
if tidak_dikenal:
votes["Layanan Kursi"] = "NO"
alasan_gagal = alasan_gagal or f"Kursi {', '.join(tidak_dikenal)} tidak dikenal"
catat("PREPARE", f"Layanan Kursi: kursi {', '.join(tidak_dikenal)} tidak dikenal -> TOLAK", "no")
elif terisi:
votes["Layanan Kursi"] = "NO"
alasan_gagal = alasan_gagal or (f"Kursi {', '.join(terisi)} sudah dipesan orang lain, "
f"silakan pilih kursi lain")
catat("PREPARE", f"Layanan Kursi: kursi {', '.join(terisi)} sudah terisi -> TOLAK", "no")
else:
votes["Layanan Kursi"] = "YES"
catat("PREPARE", "Layanan Kursi: kursi tersedia dan dikunci sementara -> SIAP", "yes")
Layanan Tiket
votes["Layanan Tiket"] = "YES"
catat("PREPARE", "Layanan Tiket: siap menerbitkan e-tiket -> SIAP", "yes")
---------- FASE 2: COMMIT / ABORT ----------
semua_yes = all(v == "YES" for v in votes.values())
tiket = None
if semua_yes:
keputusan = "COMMIT"
catat("COMMIT", "Seluruh layanan siap -> coordinator mengirim COMMIT", "yes")
DOMPET["saldo"] -= total
catat("COMMIT", f"Layanan Pembayaran: saldo dipotong Rp{total:,}, "
f"sisa Rp{DOMPET['saldo']:,}", "yes")
for k in daftar_kursi:
node.kursi[k] = nama
catat("COMMIT", f"Layanan Kursi: kursi {', '.join(daftar_kursi)} resmi atas nama {nama}", "yes")
tiket = "TIX-" + "".join(random.choices(string.ascii_uppercase + string.digits, k=6))
catat("COMMIT", f"Layanan Tiket: e-tiket {tiket} diterbitkan", "yes")
node.catat(f"Pemesanan {', '.join(daftar_kursi)} a.n. {nama} berhasil (COMMIT)")
else:
keputusan = "ABORT"
catat("ABORT", "Ada layanan yang menolak -> coordinator membatalkan seluruh transaksi", "no")
catat("ABORT", "Semua perubahan dibatalkan: saldo utuh, kursi tetap kosong, "
"tiket tidak diterbitkan", "no")
node.catat(f"Pemesanan a.n. {nama} dibatalkan (ABORT)")
RIWAYAT.insert(0, {
"waktu": time.strftime("%H:%M:%S"),
"keputusan": keputusan,
"ringkas": f"{nama} · {node.nama} · kursi {', '.join(daftar_kursi)} · Rp{total:,}",
})
del RIWAYAT[10:]
return {
"sukses": semua_yes,
"keputusan": keputusan,
"alasan": alasan_gagal,
"votes": votes,
"log": log,
"tiket": tiket,
"total": total,
"kursi": daftar_kursi,
"cabang": node.nama,
"nama": nama,
"saldo_akhir": DOMPET["saldo"],
}

4.2 Antarmuka Web Dashboard

1. Halaman Beranda

Halaman beranda merupakan tampilan awal aplikasi yang menampilkan daftar cabang bioskop beserta informasi film yang sedang diputar, jadwal penayangan, harga tiket, serta jumlah kursi yang masih tersedia. Pengguna dapat memilih salah satu cabang untuk melanjutkan proses pemesanan tiket.



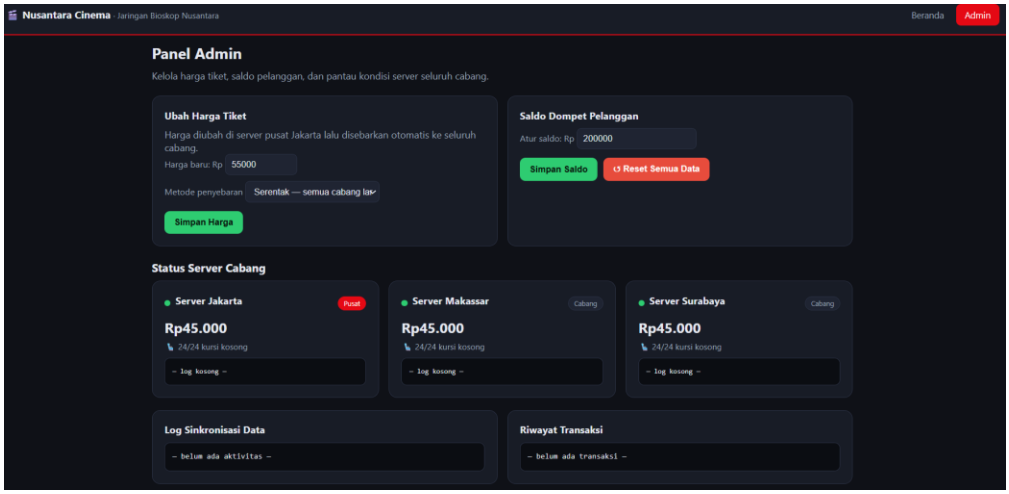
2. Halaman pemesanan

Halaman pemesanan digunakan untuk memilih kursi secara interaktif, memasukkan nama pemesan, serta mengirimkan permintaan transaksi ke server melalui protokol Two-Phase Commit (2PC). Halaman ini juga menampilkan status transaksi berhasil maupun gagal



3. Panel Admin

Panel admin digunakan untuk mengubah harga tiket pada server pusat, memilih mode replikasi (Sync atau Async), mengatur saldo pelanggan, melakukan reset sistem, serta memantau kondisi node dan riwayat transaksi secara real-time.



4.3 Pengujian Sistem

Pengujian dilakukan dengan metode Black Box untuk memastikan setiap fungsi beroperasi sesuai harapan. Tabel berikut merangkum skenario dan hasil uji:

No	Skenario Uji (Input)	Hasil yang Diharapkan	Hasil Aktual
1	Ubah harga mode Sync	Seluruh cabang langsung seragam	SESUAI
2	Ubah harga mode Async	Cabang berubah bertahap (4s & 8s)	SESUAI
3	Pesan kursi, saldo cukup	COMMIT, e-tiket diterbitkan	SESUAI
4	Pesan kursi, saldo kurang	ABORT, saldo tak terpotong	SESUAI
5	Pesan kursi sudah terisi	ABORT, kursi tetap kosong	SESUAI
6	Pesan > 6 kursi	Ditolak (maksimal 6)	SESUAI
7	Dua pemesanan bersamaan	Lock cegah kursi sama diambil	SESUAI

Bukti riwayat transaksi (dari monitor /api/admin/monitor) menunjukkan contoh berikut setelah beberapa kali pemesanan:

Waktu	Keputusan	Ringkasan Transaksi

18:40:04	Berhasil	Emil · Jakarta · kursi A1, A2 · Rp90.000
18:40:20	Berhasil	Tiara · Makassar · kursi C3 · Rp45.000
18:40:32	Berhasil	Ali · Surabaya · kursi B6 · Rp45.000
18:40:48	Dibatalkan	Rafly · Jakarta · kursi D5, D6 · Rp90.000

```
[18:40:48] DIBATALKAN · Rafly · Jakarta · kursi D5, D6 · Rp90,000
[18:40:32] BERHASIL · Ali · Surabaya · kursi B6 · Rp45,000
[18:40:20] BERHASIL · Tiara · Makassar · kursi C3 · Rp45,000
[18:40:04] BERHASIL · Emil · Jakarta · kursi A1, A2 · Rp90,000
```

Bukti log replikasi (replication log) saat admin mengubah harga ke Rp55.000 mode sync:

```
› Server pusat Jakarta menyimpan harga baru
› Seluruh cabang menerima pembaruan dan mengirim konfirmasi
› Harga kini seragam di semua cabang
```

[18:45:23] UPDATE harga = Rp55.000 · mode serentak

[18:45:23] Harga tiket diubah menjadi Rp55.000 (server pusat)

[18:45:23] Cabang Makassar menerima pembaruan harga Rp55.000 dari pusat

[18:45:23] Cabang Surabaya menerima pembaruan harga Rp55.000 dari pusat

4.4 Hasil Pengujian

Berdasarkan pengujian, seluruh skenario berjalan sesuai harapan. Sistem berhasil menjaga konsistensi data melalui replikasi Master-Slave dan menjamin atomicity transaksi melalui 2PC. Mekanisme lock terbukti mencegah dua pemesanan bersamaan mengambil kursi yang sama. Pada mode async, monitor menunjukkan cabang sempat berbeda harga sebelum akhirnya menyamai server pusat, yang merepresentasikan eventual consistency.

BAB V
PENUTUP

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, Sistem reservasi tiket bioskop Nusantara Cinema berhasil mengimplementasikan konsep sistem terdistribusi secara menyeluruh. Ketiga rumusan masalah pada BAB I telah terjawab: konsistensi data harga tiket antar cabang terjaga melalui mekanisme replikasi Master-Slave dalam dua mode, yaitu sinkron (sync) yang menjamin keseragaman data secara langsung dan asinkron (async) yang merepresentasikan eventual consistency; keutuhan (atomicity) transaksi pemesanan terjamin melalui protokol Two-Phase Commit (2PC) yang memastikan setiap transaksi hanya berakhir pada dua kemungkinan, yaitu berhasil sepenuhnya (COMMIT) atau dibatalkan sepenuhnya (ABORT); serta kondisi balapan (race condition) pada pemesanan yang datang bersamaan berhasil dicegah melalui mekanisme mutual exclusion (lock) pada proses transaksi. Hal ini diperkuat oleh hasil pengujian black box yang menunjukkan bahwa seluruh 7 skenario uji, mulai dari perubahan harga pada kedua mode replikasi, pemesanan dengan saldo cukup maupun kurang, kursi yang sudah terisi, batas maksimal kursi, hingga pemesanan bersamaan, memberikan hasil yang sesuai dengan yang diharapkan. Antarmuka web yang dibangun juga memudahkan pengguna dalam melakukan pemesanan tiket serta memudahkan admin dalam memantau kondisi node, log replikasi, dan riwayat transaksi secara real-time. Meskipun demikian, perlu ditegaskan bahwa sistem ini masih berupa simulasi yang berjalan dalam satu proses aplikasi dengan latensi jaringan yang disimulasikan melalui threading, sehingga pengembangan lebih lanjut ke arah lingkungan terdistribusi yang sesungguhnya masih diperlukan untuk memvalidasi konsep-konsep tersebut pada skala nyata.istem reservasi tiket bioskop Nusantara Cinema berhasil mengimplementasikan konsep sistem terdistribusi, meliputi replikasi data Master-

Slave (sync dan async) serta protokol Two-Phase Commit untuk menjamin atomicity transaksi. Penggunaan mutual exclusion (lock) berhasil mencegah race condition pada pemesanan bersamaan. Antarmuka web memudahkan pengguna dalam memesan tiket dan admin dalam memantau sistem.

5.2 Saran Pengembangan

- Mengembangkan node sebagai proses terpisah (multi-process) untuk simulasi nyata.
- Menambahkan penanganan kegagalan node (fault tolerance) pada protokol 2PC.
- Menerapkan basis data terdistribusi nyata seperti Cassandra atau MongoDB.
- Menambahkan autentikasi pengguna dan integrasi pembayaran digital sesungguhnya.